

Pytania i odpowiedzi

Urban Health ABM — detale parametrów, walidacji i implementacji

Dokument towarzyszący prezentacji. Odpowiada na pięć szczegółowych pytań dotyczących: parametrów FM i MM (zakresy, jednostki, wyprowadzenie), zakresu suwaków czynników ryzyka $[0, 3]$, przykładu obliczeniowego z 60-latką, rozbieżności CBR modelu vs GUS, oraz struktur danych Citizen i Zone.

Spis treści

1	Jak jest wyliczane FM i MM?	3
1.1	Definicje	3
1.1.1	FM — mnożnik płodności	3
1.1.2	MM — mnożnik śmiertelności	3
1.2	Skąd się biorą — skala bazowa	3
1.3	Dlaczego mnożnik, a nie stopa?	4
1.4	Zakres siatki w gridsearch	4
1.4.1	FM $\in [0.40, 2.50]$ z 12 punktami	4
1.4.2	MM $\in [0.30, 1.60]$ z 12 punktami	4
1.5	Interpretacja kluczowych wartości	5
1.6	Punkt stabilności znaleziony przez pełny ABM	5
2	Dlaczego czynniki ryzyka są w przedziale $[0, 3]$?	6
2.1	Suwaki RF w aplikacji Streamlit	6
2.2	Jak mnożnik działa — przykład dla palenia	6
2.3	Konkretne wartości suwaków	6
2.4	Dlaczego górna granica to 3, a nie więcej?	7
2.5	Przykład: kampania anty-nikotynowa	7
2.6	Skutek na inicjalizację RF	7
3	Przykład z 60-letnim mężczyzną — skąd wartości i dlaczego wiek bazowy 30?	8
3.1	Pełny wzór Coxa	8
3.2	Wartość po wartości	8
3.2.1	$\lambda_{0,CVD} = 6.0 \times 10^{-5}$ — bazowy hazard CVD	8
3.2.2	Współczynnik wieku: $\exp(0.06 \cdot 30) \approx 6.05$	8
3.2.3	Współczynniki $\beta = \ln(HR)$ z macierzy HAZARD_BETA	8
3.3	Pełne obliczenie krok po kroku	9
3.4	Dlaczego wiek bazowy to 30 lat?	9
3.5	Co by było, gdyby wiek bazowy był inny?	9
4	CBR walidacji: 8.30 vs 8.50 promila — skąd różnica?	10
4.1	Wartości referencyjne i modelowe	10
4.2	Dlaczego CBR jest 8.30 zamiast 8.50?	10
4.3	Dlaczego CDR jest 15.6 zamiast 13.5? (poważniejsza rozbieżność)	11
4.4	Dlaczego TFR jest niemal idealne (1.28 vs 1.26)?	11
4.5	Inżynierski wniosek	11
5	Co oznacza Dict[int, Citizen]? Co to jest Zone?	12
5.1	Dict[int, Citizen] — typowa adnotacja Pythona	12
5.1.1	Klucz = ID agenta	12
5.1.2	Wartość = obiekt Citizen	12
5.1.3	Dlaczego Dict, a nie List?	13
5.2	Klasa Zone — strefy geograficzne	13
5.2.1	Definicja klasy	13
5.2.2	Cztery strefy w modelu	13
5.2.3	Reprezentacja w SimulationEngine	13
5.2.4	Wpływ stref na model (obecnie minimalny)	14
5.3	Pełna architektura w jednym obrazie	14

1 Jak jest wyliczane FM i MM?

Co oznaczają FM i MM, skąd się biorą, dlaczego badane są w takich przedziałach? Jakie konkretne wartości testujemy?

1.1 Definicje

FM (*Fertility Multiplier*) oraz **MM** (*Mortality Multiplier*) to **bezwymiarowe mnożniki** skalujące całe tabele demograficzne z GUS Polska 2021. Nie są stopami w promilach — to mnożniki ($\times$).

1.1.1 FM — mnożnik płodności

$$P_{\text{birth, monthly}}(\text{age}) = \frac{\text{ASFR}(\text{age})}{12} \cdot \text{FM} \cdot \text{disease_reduction}$$

W kodzie (`simulation_engine.py`):

```
1 monthly_birth_prob = ASFR[wiek_kobiety] / 12 * fertility_multiplier
```

FM = 1.0 odpowiada bezpośrednio polskim danym GUS 2021 (TFR = 1.28, CBR = 8.30‰). Mnożnik różny od 1 skaluje *każdą* z 7 wiekowych stóp ASFR (15–19, 20–24, ..., 45–49 lat) tym samym współczynnikiem.

1.1.2 MM — mnożnik śmiertelności

$$P_{\text{death, monthly}} = q_x(\text{age, sex}) \cdot \text{disease_mult} \cdot \text{cox_mult} \cdot \text{MM}$$

W kodzie:

```
1 monthly_death_prob = q_x[wiek][plec] * disease_mult * cox_mult *
  mortality_multiplier
```

MM = 1.0 odpowiada Polsce 2021 (CDR = 15.6‰). MM skaluje każdą q_x z tabeli śmiertelności GUS dla każdego przedziału wiekowego i płci.

1.2 Skąd się biorą — skala bazowa

Bazowe wartości pochodzą bezpośrednio z tabel GUS:

- **ASFR** — 7 wiekowych stóp płodności (15–19, 20–24, ..., 45–49 lat); razem dają TFR = 1.28.
- q_x — 10 wiekowych prawdopodobieństw zgonu $\times$ 2 płcie z Tablic Trwania Życia 2021.

Te bazowe stopy są dane raz na zawsze. **FM i MM są jedynymi zmiennymi globalnymi**, które pozwalają eksperymentować z różnymi scenariuszami demograficznymi.

1.3 Dlaczego mnożnik, a nie stopa?

W pierwszej wersji modelu parametry nazywały się `fertility_rate` i `mortality_rate`. Były mylące — sugerowały jednostkę "stopa"(%). W rzeczywistości to bezwymiarowe mnożniki tabel wiekowych.

Stara nazwa → pomyłki:

- `fertility_rate = 2.0` → ktoś czytał "TFR = 2.0" lub "CBR = 2%". Naprawdę: $TFR \approx 2.56$, $CBR \approx 16.6\%$.
- `mortality_rate = 0.5` → wygląda jak "0.5/1000/rok". Naprawdę: "połowa polskiej śmiertelności w każdym wieku".

Po refaktorze:

- **mnożnik** — jednostka ($\times$), bezwymiarowy
- **stopa** — jednostka (%), opisuje konkretną wartość demograficzną

1.4 Zakres siatki w gridsearch

1.4.1 `FM` $\in [0.40, 2.50]$ z 12 punktami

```
FM ∈ np.linspace(0.4, 2.5, 12) = {0.40, 0.59, 0.78, 0.97, 1.16, 1.35,
                                   1.54, 1.73, 1.92, 2.12, 2.31, 2.50}
```

Dlaczego ten zakres?

- **Dolna granica 0.40** — daje $TFR \approx 0.51$, czyli skrajny scenariusz depopulacji.
- **Górna granica 2.50** — daje $TFR \approx 3.20$, przekracza wyraźnie próg zastępowalności (2.10), pozwala badać scenariusze silnego wzrostu.
- **Stabilność leży wewnątrz** — z reguły proxy $FM_{stable} = 1.88 \cdot MM$, więc dla $MM \in [0.3, 1.6]$ stabilne FM mieści się w $[0.56, 3.0]$. Wybrany zakres pokrywa większość krzywej stabilności.
- **12 punktów** — balans między rozdzielczością a kosztem obliczeniowym ($12^2 = 144$ symulacji $\times 90$ s każda).

1.4.2 `MM` $\in [0.30, 1.60]$ z 12 punktami

```
MM ∈ np.linspace(0.3, 1.6, 12) = {0.300, 0.418, 0.536, 0.655, 0.773,
                                   0.891, 1.009, 1.127, 1.245, 1.364, 1.482, 1.600}
```

Dlaczego węższy zakres niż FM?

- **Dolna granica 0.30** — skrajna redukcja śmiertelności (hipotetyczny przełom medyczny).
- **Górna granica 1.60** — 60 % wzrost śmiertelności.
- **Asymetria względem FM** — stabilność wymaga $FM > MM$ (proxy: $FM \approx 1.88 \cdot MM$). Przy $MM = 1.6$ stabilne FM byłoby już ≈ 3.0 (górna granica siatki FM), więc dalsze rozszerzanie MM nie wnosiłoby informacji.

FM	Co dzieje się z ASFR		TFR	CBR
0.5	każda $\div 2$	(kampania anty-natalistyczna)	0.64	4.2‰
1.0	bez zmian (GUS)	polska baseline	1.28	8.30‰
1.88	każda $\times 1.88$	punkt stabilności proxy	2.41	15.6‰
2.0	każda $\times 2$	podwojenie	2.56	16.6‰
2.5	każda $\times 2.5$	granica górna siatki	3.20	20.8‰
MM	Co dzieje się z q_x		CDR	Śr. dł. życia
0.3	każda $\times 0.3$	przełom medyczny	4.7‰	bardzo długie
1.0	bez zmian (GUS)	polska baseline	15.6‰	~ 77 lat
1.6	każda $\times 1.6$	granica górna siatki	25‰	znacznie krótsze

Tabela 1: Interpretacja wybranych wartości FM i MM.

1.5 Interpretacja kluczowych wartości

1.6 Punkt stabilności znaleziony przez pełny ABM

Wynik: po przeszukaniu siatki 144 punktów pełnym ABM (50 k agentów $\times$ 50 lat każdy), punkt stabilności populacji Polski to:

$$FM \approx 1.93, \quad MM \approx 1.01 \quad \implies \quad \text{score} = -0.88\%$$

Czyli Polska potrzebuje *prawie dwukrotnie wyższej płodności* niż obecna ($TFR \approx 2.47$), aby populacja przestała się kurczyć przy obecnej śmiertelności.

2 Dlaczego czynniki ryzyka są w przedziale [0, 3]?

Co oznacza `smoking = 0.3`? Co oznacza `smoking = 2.7`? Dlaczego granica górna to 3, a nie np. 5?

2.1 Suwaki RF w aplikacji Streamlit

W aplikacji `interactive_simulation_app.py` każdy z 7 czynników ryzyka ma suwak o zakresie [0.0, 3.0] z krokiem 0.1. Suwaki to **mnożniki prevalencji** — skalują bazowe prawdopodobieństwo wystąpienia danego RF u agenta przy tworzeniu populacji.

```
1 rf_multipliers[rf] = st.slider(
2     f"{rf.replace('_', ' ').title()}",
3     min_value=0.0,
4     max_value=3.0,
5     value=1.0,
6     step=0.1,
7 )
```

2.2 Jak mnożnik działa — przykład dla palenia

Bazowe prawdopodobieństwo palenia zależy od wieku agenta (kalibrowane do danych Białystoku+):

$$P_{\text{smoking, base}}(\text{age}) = \max \left(0.25 \cdot \left(1 - \frac{(\text{age} - 45)^2}{50^2} \right), 0.10 \right) \quad \text{dla } \text{age} \in [20, 70]$$

Mnożnik suwaka jest stosowany bezpośrednio:

$$P_{\text{smoking, effective}}(\text{age}) = \min (P_{\text{smoking, base}}(\text{age}) \cdot \text{slider}, 1.0)$$

2.3 Konkretnie wartości suwaków

Dla agenta w wieku szczytowej prevalencji palenia ($\text{age} = 45$, $P_{\text{base}} = 0.25$):

Suwak	Mnożnik	P efektywne (age=45)	Interpretacja
0.0	$\times 0$	0 %	całkowita eliminacja palenia (scenariusz utopijny)
0.3	$\times 0.3$	7.5 %	redukcja o 70 % (silna kampania anty-nikotynowa)
0.5	$\times 0.5$	12.5 %	redukcja o 50 %
1.0	$\times 1$	25 %	polska baseline z Białystoku+
1.5	$\times 1.5$	37.5 %	wzrost o 50 %
2.7	$\times 2.7$	67.5 %	wzrost o 170 % (skrajnie wysokie palenie)
3.0	$\times 3$	75 %	potrojenie (górna granica suwaka)

Tabela 2: Interpretacja wartości suwaka `smoking`.

2.4 Dlaczego górna granica to 3, a nie więcej?

1. **Powyżej 3 prawdopodobieństwa się nasycają.** Dla wielu wieków $P_{\text{base}} \cdot 3$ przekracza 1.0 i jest kapowane do 1.0. Dalsze zwiększanie mnożnika nie wprowadza już zmian.
2. **Skrajna eksploracja przestrzeni.** $3 \times$ baseline odpowiada hipotetycznemu społeczeństwu, w którym RF jest stale obecny u $\sim 70\%$ populacji. To wartość ekstrema dla scenariuszy "co najgorsze może się zdarzyć".
3. **Dolna granica 0.0** — pełna eliminacja (utopia zdrowia publicznego), pozwala mierzyć "potencjał interwencji– ile populacji uratujemy w pełnym scenariuszu eliminacji.
4. **Symetria interpretacyjna** — $[0, 3]$ z baseline 1.0 daje symetryczny zakres: $1/3$ do 3 (redukcja vs amplifikacja $\times 3$ w obu kierunkach).

2.5 Przykład: kampania anty-nikotynowa

Scenariusz — 70% redukcja palenia (suwak `smoking` = 0.3), pozostałe RF bez zmian.

Spodziewane efekty po 50 latach:

- Znaczący spadek prevalencji raka płuc (ponieważ $HR = 15$ dla palenia $\rightarrow$ rak płuc dominuje w determinacji ryzyka LC).
- Spadek CVD (palenie ma $HR = 2.5$ dla CVD).
- Wyższa końcowa populacja (mniej zgonów z chorób nikotyno-zależnych).
- Spadek mediany H_{cum} w populacji palaczy poniżej ~ 0.05 .

2.6 Skutek na inicjalizację RF

W `interactive_simulation_app.py` (funkcja `modified_init_rf`) suwak jest aplikowany na każdy z 7 RF analogicznie:

```
1 smoking_prob *= rf_multipliers.get("smoking", 1.0)
2 obesity_prob = min(obesity_prob, 0.45) * rf_multipliers.get("obesity", 1.0)
3 inactivity_prob *= rf_multipliers.get("physical_inactivity", 1.0)
4 alcohol_prob = base * rf_multipliers.get("alcohol_abuse", 1.0)
5 cholesterol_prob *= rf_multipliers.get("high_cholesterol", 1.0)
6 hypertension_prob *= rf_multipliers.get("hypertension_stage0", 1.0)
7 family_prob = 0.15 * rf_multipliers.get("family_history", 1.0)
```

3 Przykład z 60-letnim mężczyzną — skąd wartości i dlaczego wiek bazowy 30?

Na slajdzie jest przykład obliczeniowy: 60-letni mężczyzna z paleniem, cholesterolem i nadciśnieniem. Skąd biorą się konkretne liczby? Czy wiek bazowy to 30 lat? Dlaczego akurat 30?

3.1 Pełny wzór Coxa

$$\Delta h_d(a, X) = \lambda_{0,d} \cdot \exp(\gamma_{\text{age},d} \cdot (a - 30)) \cdot \exp\left(\sum_{i=1}^7 \beta_{d,i} \cdot X_i\right)$$

3.2 Wartość po wartości

Wszystkie liczby pochodzą z macierzy zdefiniowanych w `disease_model.py` (linie 46–78):

3.2.1 $\lambda_{0,\text{CVD}} = 6.0 \times 10^{-5}$ — bazowy hazard CVD

Z `BASELINE_HAZARD["CVD"]`. Jest to miesięczne ryzyko inicjacji CVD u 30-letniej osoby bez żadnego RF. Wartość skalibrowana do danych WHO/ESC tak, aby populacyjna prevalencja CVD zgadzała się z polską (~35 % dorosłej populacji).

3.2.2 Współczynnik wieku: $\exp(0.06 \cdot 30) \approx 6.05$

Z `AGE_HAZARD_GROWTH["CVD"] = 0.06` oraz $a - 30 = 30$ (bo agent ma 60 lat, a wiek bazowy to 30).

Model Gompertza:

$$\lambda_{\text{CVD}}(a) = \lambda_{0,\text{CVD}} \cdot \exp(0.06 \cdot (a - 30))$$

Dla $a = 60$: $\exp(0.06 \cdot 30) = e^{1.8} \approx 6.05$. Znaczy to, że hazard CVD u 60-latka jest 6.05× wyższy niż u 30-latka (oba bez RF).

Czas podwojenia hazardu:

$$T_{\text{podwojenia}} = \frac{\ln 2}{\gamma_{\text{age}}} = \frac{0.693}{0.06} \approx 11.55 \text{ lat}$$

3.2.3 Współczynniki $\beta = \ln(\text{HR})$ z macierzy `HAZARD_BETA`

Risk Factor	HR	$\beta = \ln(\text{HR})$	Źródło
smoking	2.5	0.916	ESC, Framingham
high_cholesterol	2.0	0.693	ATP III, Framingham
hypertension	2.2	0.788	Wytyczne ESC

Tabela 3: HR i β użyte w przykładzie.

Suma wykładników: $0.916 + 0.693 + 0.788 = 2.397$, więc $\exp(2.397) \approx 10.99$.

3.3 Pełne obliczenie krok po kroku

$$\Delta h = \underbrace{6.0 \cdot 10^{-5}}_{\lambda_0} \cdot \underbrace{6.05}_{\text{wiek}} \cdot \underbrace{10.99}_{\text{RF}}$$

$$\approx 3.99 \cdot 10^{-3} \text{ /miesiąc}$$

Porównanie z baseline (60-latek bez RF):

$$\Delta h_{\text{baseline}} = 6.0 \cdot 10^{-5} \cdot 6.05 = 3.63 \cdot 10^{-4}$$

$$\text{Mnożnik} = \frac{\Delta h}{\Delta h_{\text{baseline}}} = \frac{3.99 \cdot 10^{-3}}{3.63 \cdot 10^{-4}} \approx 11,0\times$$

Z aproksymacji Poissona:

$$P(\text{onset CVD w tym miesiącu}) = 1 - e^{-\Delta h} \approx 0.40 \%$$

W skali roku: $1 - (1 - 0.004)^{12} \approx 4.7 \%$.

3.4 Dlaczego wiek bazowy to 30 lat?

Wybór 30 lat jako wieku referencyjnego a_{ref} w modelu Gompertza:

$$\lambda(a) = \lambda_0 \cdot \exp(\gamma_{\text{age}} \cdot (a - a_{\text{ref}}))$$

1. **Konwencja epidemiologiczna.** W aktuarialnej i epidemiologicznej praktyce 30 lat jest standardowym wiekiem referencyjnym dla chronic diseases. Większość badań kohortowych (Framingham, MRFIT) zaczyna pomiary między 25 a 35 r.ż.
2. **Przed 30 r.ż. chronic disease hazard jest pomijalny.** Dominują przyczyny niezwiązane z RF: wypadki, samobójstwa, zakażenia. Dopiero po 30 r.ż. Gompertz exponential growth zaczyna dominować.
3. **Numerycznie wygodny.** $a - 30$ zachowuje stosunkowo niewielkie wartości wykładnika dla typowych wieków agentów (do $a = 100$, $a - 30 = 70$, $\exp(0.06 \cdot 70) = e^{4.2} \approx 67$ — jeszcze poniżej numerycznego overflow).
4. **Symetria interpretacyjna.** Średnia długość życia człowieka to ~ 78 – 82 lata; wiek 30 to $\sim 1/3$ życia, dobrze reprezentuje "młodszy dorosły".
5. **Brak hazardu u dzieci.** W kodzie dodatkowo:

$$\text{effective_age} = \max(a, 18)$$

Dzięki temu agenci < 18 r.ż. nie kumulują H_{cum} — biologicznie poprawne (chronic diseases nie rozwijają się u dzieci poza specyficznymi syndromami).

3.5 Co by było, gdyby wiek bazowy był inny?

Wartość $a_{\text{ref}} = 30$ jest *arbitralna* matematycznie — ma sens fizyczny, ale model jest *niezmienniczy* na zmianę punktu referencyjnego, o ile odpowiednio przeskalujemy λ_0 . To znaczy: jeśli zmienimy $a_{\text{ref}} = 20$, ale jednocześnie pomnożymy λ_0 przez $\exp(\gamma_{\text{age}} \cdot 10)$, dostaniemy identyczne hazardy. Konwencja 30 jest po prostu czytelna i powszechna.

4 CBR walidacji: 8.30 vs 8.50 promila — skąd różnica?

W tabeli walidacji jest napisane $\text{CBR} = 8.30 \text{ ‰}$ dla modelu, a u Polski 8.50 ‰ . Skąd ta drobna różnica? Czy to błąd?

4.1 Wartości referencyjne i modelowe

Wskaźnik	Polska 2021 (GUS)	Model (FM=MM=1.0)	Zgodność
CBR	8.50‰	8.30‰	97.6 % (drobne niedoszacowanie)
CDR	13.50‰	15.60‰	115 % (zawyżone, patrz 4.3)
TFR	1.26	1.28	101.6 % (zgodność wzorcowa)

Tabela 4: Porównanie modelu z danymi GUS Polska 2021.

4.2 Dlaczego CBR jest 8.30 zamiast 8.50?

Kilka przyczyn drobnej różnicy (0.2‰, czyli 2.4 % wartości):

1. **Zaokrąglenia w tabeli ASFR.** Wartości ASFR w naszym kodzie są zaokrąglone do trzech miejsc po przecinku:

```
1 DEFAULT_FERTILITY_TABLE = {
2     15: 0.011, 20: 0.041, 25: 0.081,
3     30: 0.082, 35: 0.034, 40: 0.007,
4     45: 0.0003,
5 }
```

GUS publikuje wartości z większą precyzją; te zaokrąglenia kumulują się do $\sim 1 \text{ ‰}$ niedoszacowania TFR (1.28 vs 1.26 — już sam ten brak skali pokrywa większość różnicy).

2. **Dyskretyzacja miesięczna.** Polski roczny ASFR jest dzielony na 12, ale empirycznie urodzenia nie rozkładają się równomiernie (są lekko sezonowe). To wprowadza drobne odchylenie $\sim 0.1 \text{ ‰}$.
3. **Synteza populacji.** Populacja syntetyczna 50 000 agentów jest *losowana* z rozkładu wiekowego GUS. Każde uruchomienie ma inny rozkład, średnio bliski celowi, ale z wariancją.
4. **Korekta przez choroby (disease_reduction).** W modelu:

$$P_{\text{birth}} = \frac{\text{ASFR}}{12} \cdot \text{FM} \cdot \underbrace{\max(1 - 0.02 \cdot n - 0.04 \cdot \text{dis}, 0.7)}_{\geq 0.7}$$

Część kobiet ma aktywne choroby (CVD lub Lung Cancer), co lekko redukuje płodność. To efekt nieobecny w danych GUS, które dotyczą "uśrednionej" Polski.

Wniosek. Różnica 0.2‰ (2.4 %) mieści się w tolerancji modelu. $\text{CBR} = 8.30$ vs GUS 8.50 — to wartości *w pełni akceptowalne* dla symulacji demograficznej. Dokładność lepsza niż 5 % w demografii to bardzo dobry wynik.

4.3 Dlaczego CDR jest 15.6 zamiast 13.5? (poważniejsza rozbieżność)

To większa rozbieżność (15 %) i ma inną przyczynę:

Powód: Tabele q_x z GUS dotyczą *całej* polskiej populacji — włącznie z osobami przewlekle chorymi, których ryzyko zgonu jest już "wbudowane" w średnie stopy.

W naszym modelu populacja syntetyczna na starcie ma 35 % prevalencji CVD, ale nie ma populacyjnego "rezerwuaru" osób z wieloma chorobami współistniejącymi. Tabela q_x jest aplikowana do agentów, którzy są przeciętnie *zdrowsi* niż polska populacja referencyjna — a mimo to dostają pełną q_x , plus dodatkowe `cox_mult` dla aktywnych chorób. To podwójnie zwiększa mortality.

Akceptowalność: odchylenie jest świadome i udokumentowane. Ważne, że punkt stabilności (FM, MM) pozostaje wewnątrz badanego zakresu siatki.

4.4 Dlaczego TFR jest niemal idealne (1.28 vs 1.26)?

$TFR = \sum ASFR \cdot \Delta age$ — to czysta suma wartości tabelarycznych, bez wpływu struktury wiekowej populacji ani innych efektów dynamicznych. Mała różnica (1.6 %) wynika tylko z zaokrągleń tabeli ASFR.

4.5 Inżynierski wniosek

W demograficznym ABM dokładność lepsza niż 5 % dla CBR i niż 20 % dla CDR jest *bardzo dobra*. Model:

- Replikuje TFR z dokładnością <2 % (idealnie).
- Replikuje CBR z dokładnością <3 % (bardzo dobrze).
- Replikuje CDR z dokładnością 15 % (akceptowalnie — z udokumentowanym powodem).

5 Co oznacza Dict[int, Citizen]? Co to jest Zone?

W pseudokodzie i diagramie architektury pojawiają się Dict[int, Citizen] oraz klasa Zone. Co to dokładnie znaczy?

5.1 Dict[int, Citizen] — typowa adnotacja Pythona

Jest to **adnotacja typu** (type hint) z modułu typing. Oznacza:

Dict[int, Citizen] = *słownik*, w którym:

- **klucz** jest typu int (liczba całkowita)
- **wartość** jest typu Citizen (obiekt klasy Citizen)

W kontekście modelu, to jest *rejestr populacji*:

```
1 class SimulationEngine:
2     def __init__(self, ...):
3         self.citizens: Dict[int, Citizen] = {}
```

Przykład zawartości:

```
1 self.citizens = {
2     1: Citizen(id=1, sex="male", age_months=384, alive=True, ...),
3     2: Citizen(id=2, sex="female", age_months=252, alive=True, ...),
4     3: Citizen(id=3, sex="female", age_months=816, alive=False, ...),
5     ...
6     50000: Citizen(id=50000, ...),
7 }
```

5.1.1 Klucz = ID agenta

Każdy obywatel ma unikalne ID generowane przez zmienną klasową Citizen._id_counter, automatycznie inkrementowaną przy tworzeniu obiektu. ID jest niezmiennie przez całe życie agenta i nie powtarza się nawet po jego śmierci.

5.1.2 Wartość = obiekt Citizen

Każdy obiekt klasy Citizen zawiera kompletny stan agenta:

```
1 class Citizen:
2     id: int # unikalny identyfikator
3     sex: str # "male" lub "female"
4     age_months: int # wiek w miesiacach (nie latach!)
5     alive: bool # True/False
6     household_id: int # przynaleznosc do gospodarstwa
7     zone_id: int # strefa geograficzna (1-4)
8     diseases: Dict[str, int] # {"CVD": 0, "Lung Cancer": 1}
9     risk_factors: Dict[str, int] # 7 RF: {"smoking": 0, ...}
0     cumulative_hazard: Dict[str, float] # pamiec Coxa
1     disability_score: float # suma wag chorob
```

5.1.3 Dłaczego Dict, a nie List?

Kryterium	Dict[int, Citizen]	List[Citizen]
Lookup po ID	$O(1)$	$O(n)$
Usunięcie martwych	nie trzeba (flaga <code>alive</code>)	drogie przesunięcia
Iteracja po żywych	<code>if c.alive</code>	analogicznie
Nowe narodziny	<code>citizens[id] = newborn</code>	<code>append()</code>
Stabilność referencji	stałe ID	indeksy się przesuwają

Tabela 5: Porównanie struktur danych dla populacji.

Kluczowe: martwi agenci *nie są usuwani* ze słownika. Mają flagę `alive = False`. Dzięki temu:

- gospodarstwo może sprawdzić czy członek żyje bez `KeyError`,
- statystyki retrospektywne są możliwe (np. ile osób zmarło w wieku 60–70 lat),
- unika się kosztownej przebudowy słownika w trakcie symulacji.

5.2 Klasa Zone — strefy geograficzne

Zone reprezentuje **strefę geograficzną** z parametrami środowiskowymi. Model ma 4 strefy, każda z własnymi parametrami.

5.2.1 Definicja klasy

```
1 class Zone:
2     id: int # identyfikator (1-4)
3     air_quality: float # 0.0 (zlej) - 1.0 (doskonalej)
4     healthcare_access: float # 0.0 - 1.0, dostepnosc opieki
5     density: int # gestosc zaludnienia (osob/km^2)
```

5.2.2 Cztery strefy w modelu

Strefa	Jakość powietrza	Dostęp do opieki	Gęstość (osób/km ²)
1	0.75 (wysoka)	0.90 (dobry)	4000
2	0.65 (średnia)	0.85	6000
3	0.70	0.80	5000
4	0.55 (niska)	0.75 (ograniczony)	8000

Tabela 6: Parametry 4 stref modelu.

Każdy agent ma atrybut `zone_id` wskazujący na jedną z tych stref. Gospodarstwa domowe (Household) również należą do stref.

5.2.3 Reprezentacja w SimulationEngine

```
1 class SimulationEngine:
2     def __init__(self, ...):
3         self.citizens: Dict[int, Citizen] = {}
4         self.households: Dict[int, Household] = {}
```

```

5     self.zones: Dict[int, Zone] = {
6         1: Zone(id=1, air_quality=0.75, ...),
7         2: Zone(id=2, air_quality=0.65, ...),
8         3: Zone(id=3, air_quality=0.70, ...),
9         4: Zone(id=4, air_quality=0.55, ...),
10    }

```

5.2.4 Wpływ stref na model (obecnie minimalny)

Status w obecnej wersji: parametry środowiskowe stref są *przechowywane*, ale ich wpływ na śmiertelność i choroby jest *minimalny*. Głównym motorem demograficznym są wiekowe tabele GUS (zgodnie z dokumentacją projektu).

Potencjalne rozszerzenie: kalibrowane wagi mogłyby aktywować różnice między strefami:

- Strefa 4 (wysoka gęstość, niska jakość powietrza) — wyższa baza dla **smoking, respiratory disease**.
- Strefa 1 (przedmieścia, niska gęstość) — niższa baza dla **air-pollution-related** ryzyka.

W obecnej wersji projekt skupia się na demografii bazowej + 7 RF (model Coxa) jako głównym driverze.

5.3 Pełna architektura w jednym obrazie

```

1 SimulationEngine
2 |-- citizens: Dict[int, Citizen] # 50 000 obywateli, klucz = id
3 |-- households: Dict[int, Household] # rodziny, klucz = id
4 |-- zones: Dict[int, Zone] # 4 strefy
5 |-- disease_model: DiseaseModel # parametry Coxa
6 |-- yearly_stats: Dict[int, Dict] # statystyki per rok
7
8 Citizen
9 |-- id, sex, age_months, alive
10 |-- household_id -> klucz do households
11 |-- zone_id -> klucz do zones
12 |-- diseases: Dict[str, int]
13 |-- risk_factors: Dict[str, int] # 7 RF
14 |-- cumulative_hazard: Dict[str, float] # pamięć Coxa
15
16 Household
17 |-- id, zone_id
18 |-- members: List[int] # lista ID agentów (nie obiektów!)
19
20 Zone
21 |-- id, air_quality, healthcare_access, density

```

Ważny szczegół: `Household.members` przechowuje *listę ID* obywateli, nie referencji do obiektów. To zapobiega cyklicznym zależnościom i ułatwia serializację (np. zapis do JSON).